

Add Two Numbers

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

$l_1 : 2 \rightarrow 4 \rightarrow 3$

$l_2 : 5 \rightarrow 6 \rightarrow 4$

$2 \rightarrow 4 \rightarrow 3 \Rightarrow 342$

$5 \rightarrow 6 \rightarrow 4 \Rightarrow 465$

$$342 + 465 = 807$$

ans: $7 \rightarrow 0 \rightarrow 8$

$l_1 : 9 \rightarrow 9 \rightarrow 9 \rightarrow 9 \rightarrow 9 \rightarrow 9 \rightarrow 9$

$l_2 : 9 \rightarrow 9 \rightarrow 9 \rightarrow 9$

$$\begin{array}{r} \\ \\ \\ + \\ \hline 10 \ 009998 \end{array}$$

ans: $8 \rightarrow 9 \rightarrow 9 \rightarrow 9 \rightarrow 9 \rightarrow 0 \rightarrow 0 \rightarrow 0 \rightarrow 1$

Brute force

1. Convert both ll to numbers
2. Add the numbers
3. Convert the sum to ll

TC: number of nodes in $l_1 = n$
number of nodes in $l_2 = m$

Traverse $n+m$ nodes
 $O(n+m)$

SC: $n+m$ sized (ll)
 $O(n+m)$

Optimized

1. Traverse both lists simultaneously
2. Add digits + carry
3. Create result ll on the go

$l_1: 2 \rightarrow 4 \rightarrow 3$
 $l_2: 5 \rightarrow 6 \rightarrow 4$

$l_1:$	2		4		3
$l_2:$	5		6		4
carry:	0		0		1
Sum:	7	$\longrightarrow$	10	$\longrightarrow$	8
digit:			0		
carry:			1		

$7 \rightarrow 0 \rightarrow 8$

$l_1 : 9 \rightarrow 9 \rightarrow 9 \rightarrow 9 \rightarrow 9 \rightarrow 9 \rightarrow 9$
 $l_2 : 9 \rightarrow 9 \rightarrow 9 \rightarrow 9$
 $sum : 18 \quad 19 \quad 19 \quad 19 \quad 10 \quad 10 \quad 10$
 $d : 8 \quad 9 \quad 9 \quad 9 \quad 0 \quad 0 \quad 0 \quad 1$
 $c : 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1 \quad 1$

$8 \rightarrow 9 \rightarrow 9 \rightarrow 0 \rightarrow 0 \rightarrow 0 \rightarrow 1$

```

ListNode addTwoNumbers(ListNode l1, ListNode l2) {
    ListNode dummy = new ListNode(-1);
    ListNode cur = dummy;
    carry = 0;
    while (l1 != null || l2 != null || carry != 0)
    {
        sum = carry;
        if (l1 != null) {
            sum += l1.val;
            l1 = l1.next;
        }
        if (l2 != null) {
            sum += l2.val;
            l2 = l2.next;
        }
    }
}

```

```
int digit = sum%10  
carry = sum/10
```

```
curr.next = new LN(digit);
```

```
curr = curr.next;
```

```
}
```

```
return dummy.next;
```

```
}
```

$l_1 \rightarrow u$

$l_2 \rightarrow m$

TC: $O(\text{Max}(u, m))$

SC: $O(\text{Max}(u, m))$